INSTITUTO INFNET

ESCOLA SUPERIOR DE TECNOLOGIA

GRADUAÇÃO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



Teste de Performance - TP1

Estruturas de Dados e Algoritmos Avançados

ALUNO: CARLOS MATHEUS MONTEIRO DO NASCIMENTO

PROF(A): RICARDO PIRES MESQUITA

E-MAIL: carlos.dnascimento@al.infnet.edu.br

Sumário

Sumário **1**
Introdução **2**
Estrutura do projeto 2
Por que o utils.py? 2
utils.py **2**
Funções de apoio 5
Exercício 01 **6**
O problema: fila de suporte técnico com prioridades 6
Por que uma heap binária 6
Comparação com outras estruturas 7
Análise em Big O das operações críticas 7
Exercício 02 **8**
Funções auxiliares de índice 8
Invariantes 8
Exercício 03 – Inserção (insert) **8**
Exercício 04 – Remoção da raiz (extract_max) **10**
Exercício 05 – Busca linear (contains) **12**
Exercício 06 – Remoção de elemento arbitrário (delete) **13**
Exercício 07 – Validação estrutural (is_valid_heap) **15**
Exercício 08 – Construção a partir de array (build_heap) **16**
Exercício 09 – build_heap × inserção incremental **19**
Exercício 10 – Heap sort parcial (k maiores) **21**
Exercício 11 – Análise de complexidade empírica **22**
Exercício 12 – Documentação técnica **26**
Visão Geral da Arquitetura 26
Representação Interna: Array de Nível 26
Instrumentação de Passos 27
Operações e Decisões Algorítmicas 27
Relação das Decisões com as Propriedades Formais da Heap 28
LINK REPO **29**

Introdução

Neste trabalho eu implementei uma heap binária de máximo em Python, do zero, usando orientação a objetos. A mesma heap é usada em todos os exercícios: cada um pega uma operação (inserir, remover, buscar, construir a partir de um array, e assim por diante), mostra ela funcionando e analisa o custo.

Estrutura do projeto

Os exercícios 1 e 2 são teóricos e estão escritos neste documento. O exercício 12 é a documentação técnica, também consta nesse documento. Do 3 ao 11 os exercícios têm código, e cada um fica em uma pasta própria com um arquivo `execucao.txt` onde ficam suas execuções.

Por que o utils.py?

Toda a lógica da heap fica na classe `BinaryHeap`, dentro do `utils.py`. Separei assim para não repetir código: os exercícios só importam a classe e usam. No mesmo arquivo ficam algumas funções que ajudam nos testes de desempenho: `generate_vector` gera arrays em ordem crescente, decrescente, aleatória ou quase ordenada; `benchmark` roda uma medição para várias combinações (padrão de entrada × tamanho) e tira a média; e `print_table` imprime o resultado em tabela.

Nas seções a seguir, cada exercício traz as partes principais do código (sem os imports; o código completo está no repositório) e, logo abaixo, a saída da execução.

utils.py

```
class BinaryHeap:
    def __init__(self, data: list = None):
        self.__data = list(data) if data is not None else []
        self.__comparisons = 0
        self.__swaps = 0
    def reset_counters(self):
        self.__comparisons = 0
        self.__swaps = 0
    @property
    def data(self) -> list:
        return self.__data.copy()
    @property
    def comparisons(self) -> int:
        return self.__comparisons
    @property
```

```
    def swaps(self) -> int:
        return self.__swaps
    @property
    def steps(self) -> int:
        return self.__comparisons + self.__swaps
    def __parent(self, i: int) -> int:
        return (i - 1) // 2
    def __left(self, i: int) -> int:
        return 2 * i + 1
    def __right(self, i: int) -> int:
        return 2 * i + 2
    def __sift_up(self, i: int) -> list:
        # Sobe o elemento da posição i enquanto ele for maior que o pai.
        swapped = []
        while i > 0:
            parent = self.__parent(i)
            self.__comparisons += 1
            if self.__data[i] > self.__data[parent]:
                swapped.append((i, parent))
                self.__data[i], self.__data[parent] = self.__data[parent],
self.__data[i]
                self.__swaps += 1
                i = parent
            else:
                break
        return swapped
    def __sift_down(self, i: int) -> list:
        # Desce o elemento da posição i trocando com o maior filho até
        # nenhum filho ser maior que ele.
        swapped = []
        n = len(self.__data)
        while True:
            left = self.__left(i)
            right = self.__right(i)
            maximum = i
            if left < n:
                self.__comparisons += 1
                if self.__data[left] > self.__data[maximum]:
                    maximum = left
            if right < n:
                self.__comparisons += 1
```

```
                if self.__data[right] > self.__data[maximum]:
                    maximum = right
            if maximum == i:
                break
            swapped.append((i, maximum))
            self.__data[i], self.__data[maximum] = self.__data[maximum],
self.__data[i]
            self.__swaps += 1
            i = maximum
        return swapped
    def insert(self, value) -> list:
        self.__data.append(value)
        return self.__sift_up(len(self.__data) - 1)
    def extract_max(self) -> tuple:
        if not self.__data:
            return None, []
        if len(self.__data) == 1:
            return self.__data.pop(), []
        maximum = self.__data[0]
        self.__data[0] = self.__data.pop()
        return maximum, self.__sift_down(0)
    def contains(self, value) -> tuple:
        visited = []
        for i, v in enumerate(self.__data):
            visited.append(i)
            self.__comparisons += 1
            if v == value:
                return True, visited
        return False, visited
    def delete(self, value) -> tuple:
        index = -1
        for i, v in enumerate(self.__data):
            self.__comparisons += 1
            if v == value:
                index = i
                break
        if index == -1:
            return False, []
        if index == len(self.__data) - 1:
            self.__data.pop()
            return True, []
```

```
        removed_value = self.__data[index]
        self.__data[index] = self.__data.pop()
        replacement = self.__data[index]
        self.__comparisons += 1  # comparação entre substituto e removido
        if replacement > removed_value:
            return True, self.__sift_up(index)
        return True, self.__sift_down(index)
    @classmethod
    def build_heap(cls, array: list, snapshots: bool = True) ->
tuple["BinaryHeap", list]:
        # snapshots guarda o array após cada sift-down; cada cópia custa
O(n).
        heap = cls(array)
        n = len(heap.__data)
        log = []
        for i in range((n - 2) // 2, -1, -1):
            swapped = heap.__sift_down(i)
            log.append((i, swapped, list(heap.__data) if snapshots else
None))
        return heap, log
    @classmethod
    def is_valid_heap(cls, array: list) -> tuple:
        heap = cls(array)
        n = len(heap.__data)
        for i in range(n):
            left = heap.__left(i)
            right = heap.__right(i)
            if left < n and heap.__data[left] > heap.__data[i]:
                return False, (i, left)
            if right < n and heap.__data[right] > heap.__data[i]:
                return False, (i, right)
        return True, None

# Funções de apoio aos testes: explicadas na seção seguinte;
# o código delas está no utils.py do repositório.
```

Funções de apoio

Além da classe, o `utils.py` tem quatro funções que não fazem parte da heap. Elas existem só para os testes de desempenho (exercícios 03, 08, 09 e 11). Sem elas, cada um desses exercícios teria que gerar os próprios arrays, repetir os laços de medição e montar a tabela na mão.

- `generate_vector(n, pattern)`: gera um array de tamanho n em um de quatro padrões: `sorted` (crescente), `reverse` (decrescente), `random` (aleatório) e `nearly_sorted` (quase ordenado). Os padrões existem porque o custo da heap depende da ordem da entrada: crescente é o pior caso da inserção, decrescente é o melhor, e os outros dois ficam no meio.
- `generate_nearly_sorted(n)`: usada pela função anterior. Parte de um array ordenado e troca de lugar alguns pares aleatórios (cerca de 5% de n), simulando dados que já chegam quase em ordem, algo comum na prática.
- `benchmark(axes, measure, metrics, samples)`: roda uma função de medição para todas as combinações dos eixos (por exemplo, cada padrão com cada tamanho de n) e devolve as linhas da tabela. Com `samples` maior que 1, repete cada medição e tira a média, o que suaviza a variação dos arrays aleatórios e dos tempos.
- `print_table(title, rows, headers)`: imprime essas linhas no terminal em formato de tabela, usando a biblioteca `tabulate`. É ela que gera as tabelas que aparecem nas execuções.

Os exercícios que usam arrays aleatórios fixam `random.seed(42)`, então os números saem iguais a cada execução e dá para comparar uma nova rodada com o `execucao.txt` registrado.

Exercício 01

O problema: fila de suporte técnico com prioridades

Um sistema de helpdesk recebe tickets de suporte o tempo todo. Cada ticket tem um nível de urgência, de 1 (baixo) a 5 (crítico), e o atendente sempre deve resolver o ticket mais urgente disponível. Novos tickets chegam a qualquer momento, e um ticket pode ter a urgência aumentada enquanto espera.

As operações que o sistema precisa são:

- `inserir(ticket, urgência)`: um novo ticket chega na fila;
- `atender()`: remove e devolve o ticket mais urgente;
- `consultar()`: mostra qual é o próximo, sem remover;
- `atualizar(ticket, nova_urgência)`: aumenta a urgência de um ticket que está esperando.

Por que uma heap binária

Uma heap de máximo guarda os tickets em um array em que o mais urgente está sempre na posição 0, a raiz. A invariante `A[i] >= A[2i+1]` e `A[i] >= A[2i+2]` é mantida a cada inserção e remoção pelas operações de sift-up e sift-down. Isso garante:

- `consultar()` em O(1): a raiz é sempre o máximo, sem precisar percorrer nada;

- `inserir()` e `atender()` em O(log n): a árvore é quase completa, então a altura é log2 n;
- Implementação com um array contíguo: sem ponteiros, sem fragmentação de memória e com bom uso de cache.

O padrão de uso do helpdesk, com muitas inserções e extração frequente do máximo, é exatamente o que a heap faz bem.

Comparação com outras estruturas

Comparei a heap com duas alternativas: um array mantido ordenado e uma fila simples (FIFO).

Operação	Heap binária	Array ordenado	Fila FIFO
inserir	O(log n)	O(n)	O(1)
atender	O(log n)	O(1)	O(1)
consultar	O(1)	O(1)	O(1)
atualizar	O(log n)*	O(n)	O(1)
respeita a urgência?	sim	sim	não

* Com um mapa auxiliar guardando a posição de cada ticket no array.

Array ordenado: Mantém os tickets já em ordem de urgência, então atender e consultar custam O(1): o maior está sempre na última posição. O problema é a inserção em O(n), porque é preciso deslocar elementos para abrir espaço, e atualizar também exige busca e reordenação. Com 10.000 tickets, uma única inserção pode deslocar até 10.000 elementos, o que inviabiliza o uso quando os tickets chegam com frequência.

Fila simples (FIFO): Atende na ordem de chegada. Todas as operações são O(1), mas a fila ignora completamente a urgência: um ticket crítico que chega depois de 50 tickets de baixa prioridade só é atendido depois de todos eles, o que é inaceitável num sistema de suporte.

Análise em Big O das operações críticas

Seja n o número de tickets na fila.

- `inserir()` **O(log n):** O ticket entra no final do array (a próxima folha da árvore) e sobe (sift-up), trocando com o pai enquanto for mais urgente que ele.

  No pior caso o ticket sobe até a raiz, ou seja, no máximo log2 n trocas.

- `atender()` **O(log n):** Remove a raiz, coloca a última folha no lugar e faz ela descer (sift-down), trocando com o filho mais urgente.

  No pior caso o elemento desce todos os log2 n níveis.

- `consultar()` **O(1):** É só ler a raiz, sem nenhuma comparação.

- `atualizar()` **O(log n):** Com um mapa auxiliar `index[ticket]` que guarda a posição de cada ticket no array, achar o ticket custa O(1). Depois, se a urgência aumentou, ele sobe; se diminuiu, ele desce.

  Nos dois casos o ticket percorre no máximo log2 n níveis.

Exercício 02

A heap binária máxima está implementada na classe `BinaryHeap`, no `utils.py` (o código está na seção utils.py, no início deste documento). Nenhuma outra estrutura é usada para guardar os elementos.

Funções auxiliares de índice

A posição de pai e filhos é calculada a partir do índice `i` de cada nó.

Método	Fórmula	Retorna
`__parent(i)`	`(i - 1) // 2`	índice do pai
`__left(i)`	`2 * i + 1`	índice do filho esquerdo
`__right(i)`	`2 * i + 2`	índice do filho direito

Invariantes

Uma heap binária máxima é válida quando, para todo índice `i` que tem filhos:

- `data[i] >= data[2*i + 1]` (filho esquerdo);
- `data[i] >= data[2*i + 2]` (filho direito).

Além disso, a árvore é sempre quase completa: todos os níveis ficam cheios, menos talvez o último, que é preenchido da esquerda para a direita. Isso é garantido pelo uso sequencial da lista: `append` coloca a próxima folha e `pop` remove a última.

A verificação dessas invariantes está no método `BinaryHeap.is_valid_heap`, usado no exercício 07.

Exercício 03 – Inserção (insert)

`ex03/__main__.py`

```
def run_sequence(name: str, values: list) -> None:
    heap = BinaryHeap()
    total = 0
    print(f"\n{name}: {values}")
    for value in values:
        swapped = heap.insert(value)
```

```
        total += len(swapped)
        print(f"insert({value:2d}) -> trocas: {swapped}")
    print(f"total de trocas: {total}")

def measure(pattern: str, n: int) -> tuple:
    heap = BinaryHeap()
    total = 0
    for value in generate_vector(n, pattern):
        total += len(heap.insert(value))
    return total, total / n, math.log2(n)

# Pior caso: sequência crescente força sift-up máximo em cada inserção.
# O novo elemento é sempre o maior, sobe até a raiz percorrendo todos os
# níveis. Para n elementos, cada inserção custa O(log n), totalizando
# O(n log n) trocas na sequência inteira.
# Melhor caso: sequência decrescente, nenhum elemento precisa subir,
# custo O(1) por inserção, O(n) no total.
run_sequence("Crescente", [10, 20, 30, 40, 50])
run_sequence("Decrescente", [50, 40, 30, 20, 10])
run_sequence("Aleatória", [30, 10, 50, 20, 40])

# Em escala: no padrão sorted, trocas/inserção cresce com log2 n (pior
caso);
# em reverse fica em 0 (melhor caso); random e nearly_sorted ficam entre os
dois.
rows, headers = benchmark(
    axes={"padrão": ["sorted", "reverse", "random", "nearly_sorted"],
          "n": [10, 100, 1000, 10000]},
    measure=measure,
    metrics=["trocas", "trocas/inserção", "log2 n (altura)"],
    samples=5,
)
print_table("Trocas na inserção por padrão de entrada (média de 5
execuções)",
            rows, headers, floatfmt=".2f")
```

Execução (`ex03/execucao.txt`):

```
$ python -m ex03
Crescente: [10, 20, 30, 40, 50]
insert(10) -> trocas: []
insert(20) -> trocas: [(1, 0)]
insert(30) -> trocas: [(2, 0)]
```

```
insert(40) -> trocas: [(3, 1), (1, 0)]
insert(50) -> trocas: [(4, 1), (1, 0)]
total de trocas: 6

Decrescente: [50, 40, 30, 20, 10]
insert(50) -> trocas: []
insert(40) -> trocas: []
insert(30) -> trocas: []
insert(20) -> trocas: []
insert(10) -> trocas: []
total de trocas: 0

Aleatória: [30, 10, 50, 20, 40]
insert(30) -> trocas: []
insert(10) -> trocas: []
insert(50) -> trocas: [(2, 0)]
insert(20) -> trocas: [(3, 1)]
insert(40) -> trocas: [(4, 1)]
total de trocas: 3
Trocas na inserção por padrão de entrada (média de 5 execuções)
| padrão         |     n |    trocas |   trocas/inserção |   log2 n (altura) |
|----------------|-------|-----------|-------------------|-------------------|
| sorted         |    10 |     19.00 |              1.90 |              3.32 |
| sorted         |   100 |    480.00 |              4.80 |              6.64 |
| sorted         |  1000 |   7987.00 |              7.99 |              9.97 |
| sorted         | 10000 | 113631.00 |             11.36 |             13.29 |
| reverse        |    10 |      0.00 |              0.00 |              3.32 |
| reverse        |   100 |      0.00 |              0.00 |              6.64 |
| reverse        |  1000 |      0.00 |              0.00 |              9.97 |
| reverse        | 10000 |      0.00 |              0.00 |             13.29 |
| random         |    10 |      6.20 |              0.62 |              3.32 |
| random         |   100 |    108.00 |              1.08 |              6.64 |
| random         |  1000 |   1243.40 |              1.24 |              9.97 |
| random         | 10000 |  12678.60 |              1.27 |             13.29 |
| nearly_sorted  |    10 |     15.60 |              1.56 |              3.32 |
| nearly_sorted  |   100 |    349.20 |              3.49 |              6.64 |
| nearly_sorted  |  1000 |   4285.40 |              4.29 |              9.97 |
| nearly_sorted  | 10000 |  44389.20 |              4.44 |             13.29 |
```

Exercício 04 – Remoção da raiz (extract_max)

`ex04/__main__.py`

```python
def drain_heap(values: list) -> None:
```

```
    heap = BinaryHeap()
    for v in values:
        heap.insert(v)
    print(f"Heap inicial: {heap.data}")
    step = 1
    while True:
        maximum, swapped = heap.extract_max()
        if maximum is None:
            break
        print(f"\nPasso {step}: extract_max() -> {maximum}")
        print(f"trocas (sift-down): {swapped}")
        print(f"array resultante: {heap.data}")
        step += 1

# Complexidade de extract_max(): O(log n)
# A raiz (máximo) é removida e substituída pela última folha.
# O sift-down desce comparando o nó com seus dois filhos e trocando
# com o maior deles, repetindo até não haver filho maior.
# A árvore completa tem altura h = log2 n, portanto o sift-down
# executa no máximo log2 n trocas, ou seja, O(log n) por chamada.
# Remoções sucessivas até esvaziar a heap custam:
# log(n) + log(n-1) + ... + log(1) = O(n log n) no total.
drain_heap([40, 20, 35, 10, 15, 30, 25])
```

Execução (`ex04/execucao.txt`):

```
$ python -m ex04
Heap inicial: [40, 20, 35, 10, 15, 30, 25]

Passo 1: extract_max() -> 40
trocas (sift-down): [(0, 2), (2, 5)]
array resultante: [35, 20, 30, 10, 15, 25]

Passo 2: extract_max() -> 35
trocas (sift-down): [(0, 2)]
array resultante: [30, 20, 25, 10, 15]

Passo 3: extract_max() -> 30
trocas (sift-down): [(0, 2)]
array resultante: [25, 20, 15, 10]

Passo 4: extract_max() -> 25
```

```
trocas (sift-down): [(0, 1)]
array resultante: [20, 10, 15]

Passo 5: extract_max() -> 20
trocas (sift-down): []
array resultante: [15, 10]

Passo 6: extract_max() -> 15
trocas (sift-down): []
array resultante: [10]

Passo 7: extract_max() -> 10
trocas (sift-down): []
array resultante: []
```

Exercício 05 – Busca linear (contains)

`ex05/__main__.py`

```python
def search_heap(values):
    heap, _ = BinaryHeap.build_heap(values)
    print(f"Array: {heap.data}")
    for target in [35, 20, 99]:
        found, visited = heap.contains(target)
        status = "encontrado" if found else "não encontrado"
        print(f"\ncontains({target}): {status}")
        print(f"índices visitados: {visited}")
        print(f"comparações: {len(visited)}/{len(heap.data)}")

# Algoritmo: varredura linear do array interno.
# A propriedade de heap (pai >= filhos) não impõe ordem entre elementos
# de subárvores diferentes nem entre irmãos, portanto não há como descartar
# regiões do array sem verificá-las. A busca precisa percorrer todos os
# elementos no pior caso.
# Complexidade: O(n)
# - Melhor caso O(1): valor encontrado na raiz (índice 0).
# - Pior caso O(n): valor está na última posição ou não existe na heap.
# Por que heap não é adequada para buscas frequentes:
# Uma BST balanceada garante busca em O(log n) explorando
# a ordenação total entre nós. A heap mantém apenas a relação pai >= filho,
# sem ordenação entre irmãos, não há como decidir em qual subárvore buscar.
# Aplicações que misturam extração de máximo com buscas frequentes devem
```

```
# combinar heap com uma tabela hash (O(1) por busca) ou trocar para outra
# estrutura.
search_heap([50, 30, 40, 10, 20, 35, 25])
```

Execução (`ex05/execucao.txt`):

```
$ python -m ex05
Array: [50, 30, 40, 10, 20, 35, 25]

contains(35): encontrado
índices visitados: [0, 1, 2, 3, 4, 5]
comparações: 6/7

contains(20): encontrado
índices visitados: [0, 1, 2, 3, 4]
comparações: 5/7

contains(99): não encontrado
índices visitados: [0, 1, 2, 3, 4, 5, 6]
comparações: 7/7
```

Exercício 06 – Remoção de elemento arbitrário (delete)

`ex06/__main__.py`

```
def delete_and_show(heap: BinaryHeap, value: int) -> None:
    print(f"delete({value})")
    print(f"antes: {heap.data}")
    found, swapped = heap.delete(value)
    if not found:
        print(f"valor {value} não encontrado na heap")
        return
    print(f"trocas: {swapped}")
    print(f"depois: {heap.data}")

# Complexidade de delete(value): O(n)
# O custo dominante é a busca linear pelo valor (O(n)), pois a propriedade
# de heap não impõe ordem entre irmãos e não permite descartar subárvores.
# Após encontrar o elemento, a reorganização (sift-up ou sift-down) custa
# O(log n). O total é O(n) + O(log n) = O(n).
# Impacto no desempenho global:
```

```python
# delete() torna a heap ineficiente para remoções arbitrárias frequentes.
# Se buscas e deleções por valor forem comuns, a estrutura correta é uma
# BST balanceada (O(log n) para busca e remoção) ou heap + tabela hash
# de posições para O(log n) por deleção sem a busca linear.

VALUES = [100, 80, 90, 60, 70, 85, 88]

# Caso 1: valor existente, substituto (88) maior que o removido (70) ->
sift-up
h1, _ = BinaryHeap.build_heap(VALUES)
print("Caso 1 - valor existente (sift-up):")
delete_and_show(h1, 70)

# Caso 2: valor existente, substituto (88) menor que o removido (100) ->
sift-down
h2, _ = BinaryHeap.build_heap(VALUES)
print("\nCaso 2 - valor existente (sift-down):")
delete_and_show(h2, 100)

# Caso 3: valor inexistente
h3, _ = BinaryHeap.build_heap(VALUES)
print("\nCaso 3 - valor inexistente:")
delete_and_show(h3, 999)
```

Execução (`ex06/execucao.txt`):

```
$ python -m ex06
Caso 1 - valor existente (sift-up):
delete(70)
antes: [100, 80, 90, 60, 70, 85, 88]
trocas: [(4, 1)]
depois: [100, 88, 90, 60, 80, 85]

Caso 2 - valor existente (sift-down):
delete(100)
antes: [100, 80, 90, 60, 70, 85, 88]
trocas: [(0, 2)]
depois: [90, 80, 88, 60, 70, 85]

Caso 3 - valor inexistente:
delete(999)
antes: [100, 80, 90, 60, 70, 85, 88]
```

```
valor 999 não encontrado na heap
```

Exercício 07 – Validação estrutural (is_valid_heap)

ex07/__main__.py

```python
# Algoritmo: percorre todos os índices do array verificando, para cada nó i,
# se o filho esquerdo (2i+1) e o filho direito (2i+2) - quando existem -
# são menores ou iguais ao pai. A primeira violação encontrada encerra a
# verificação imediatamente e retorna False.
# Justificativa: como a heap é representada nível a nível no array, iterar
# linearmente cobre todas as relações pai-filho exatamente uma vez. Não é
# necessário percorrer subárvores recursivamente, os índices já fazem isso.
# Complexidade: O(n) no pior caso (array válido, todas as relações
verificadas).
# Melhor caso O(1): violação na primeira relação verificada (índices 0 e 1).

def check(description: str, array: list) -> None:
    is_valid, violation = BinaryHeap.is_valid_heap(array)
    status = "válida" if is_valid else "inválida"
    print(f"{description}")
    print(f"array: {array}")
    if violation:
        parent, child = violation
        print(f"resultado: {status} -> array[{child}]={array[child]} >
array[{parent}]={array[parent]}")
    else:
        print(f"resultado: {status}")
    print()

heap, _ = BinaryHeap.build_heap([50, 30, 40, 10, 20, 35, 25])

# Arrays válidos
check("Heap válida - construída pela classe", heap.data)
check("Heap válida - um elemento", [42])
check("Heap válida - array vazio", [])

# Arrays inválidos
check("Inválida - violação na raiz", [40, 50, 30])
check("Inválida - violação na subárvore esquerda", [50, 30, 40, 10, 45, 35,
25])
```

```
check("Inválida - violação na subárvore direita", [50, 30, 20, 10, 25, 15,
30])
```

Execução (`ex07/execucao.txt`):

```
$ python -m ex07
Heap válida - construída pela classe
array: [50, 30, 40, 10, 20, 35, 25]
resultado: válida

Heap válida - um elemento
array: [42]
resultado: válida

Heap válida - array vazio
array: []
resultado: válida

Inválida - violação na raiz
array: [40, 50, 30]
resultado: inválida -> array[1]=50 > array[0]=40

Inválida - violação na subárvore esquerda
array: [50, 30, 40, 10, 45, 35, 25]
resultado: inválida -> array[4]=45 > array[1]=30

Inválida - violação na subárvore direita
array: [50, 30, 20, 10, 25, 15, 30]
resultado: inválida -> array[6]=30 > array[2]=20
```

Exercício 08 – Construção a partir de array (build_heap)

`ex08/__main__.py`

```
# Algoritmo: percorre os nós internos de baixo para cima
# (do índice (n-2)//2 até 0) aplicando sift-down em cada um. Folhas são
# ignoradas, pois já satisfazem a propriedade trivialmente.
# Por que é O(n) e não O(n log n):
# A maioria dos nós fica nos níveis inferiores da árvore e percorre poucos
# níveis no sift-down. A soma das alturas de todos os nós de uma árvore
# completa com n elementos converge para O(n). Inserir um a um custaria
```

```python
# O(n log n) porque cada inserção pode subir até a raiz.

def demo(description: str, array: list) -> None:
    print(f"{description}")
    print(f"inicial: {array}")
    heap, log = BinaryHeap.build_heap(array)
    for i, swapped, state in log:
        print(f"sift-down i={i}: trocas={swapped} -> {state}")
    if not log:
        print("nenhum sift-down necessário (array já é heap válida)")
    is_valid, _ = BinaryHeap.is_valid_heap(heap.data)
    print(f"final: {heap.data} válida={is_valid}")
    print()

def measure(pattern: str, n: int) -> tuple:
    heap, _ = BinaryHeap.build_heap(generate_vector(n, pattern),
snapshots=False)
    is_valid, _ = BinaryHeap.is_valid_heap(heap.data)
    assert is_valid, f"build_heap gerou heap inválida ({pattern}, n={n})"
    return heap.steps, heap.swaps, heap.steps / n

demo("1 elemento", [42])
demo("2 elementos", [1, 5])
demo("Array arbitrário (7)", [5, 3, 8, 1, 7, 2, 9])
demo("Ordem crescente (7)", [1, 2, 3, 4, 5, 6, 7])
demo("Ordem decrescente (7)", [7, 6, 5, 4, 3, 2, 1])
demo("Array arbitrário (10)", [4, 10, 3, 5, 1, 8, 9, 2, 7, 6])
demo("Ordem crescente (15)", list(range(1, 16)))

# Tamanhos e padrões variados: os passos crescem na mesma proporção que n
# (n 10× maior, passos ~10× maiores), por isso passos/n fica estável,
# em torno de 3: O(n)
rows, headers = benchmark(
    axes={"padrão": ["sorted", "reverse", "random", "nearly_sorted"],
          "n": [10, 100, 1000, 10000]},
    measure=measure,
    metrics=["passos", "trocas", "passos/n"],
    samples=5,
)
print_table("build_heap por padrão de entrada (média de 5 execuções; todas
as heaps válidas)",
```

```
            rows, headers, floatfmt=".2f")
```

Execução (`ex08/execucao.txt`):

```
$ python -m ex08
1 elemento
inicial: [42]
nenhum sift-down necessário (array já é heap válida)
final: [42] válida=True


2 elementos
inicial: [1, 5]
sift-down i=0: trocas=[(0, 1)] -> [5, 1]
final: [5, 1] válida=True


Array arbitrário (7)
inicial: [5, 3, 8, 1, 7, 2, 9]
sift-down i=2: trocas=[(2, 6)] -> [5, 3, 9, 1, 7, 2, 8]
sift-down i=1: trocas=[(1, 4)] -> [5, 7, 9, 1, 3, 2, 8]
sift-down i=0: trocas=[(0, 2), (2, 6)] -> [9, 7, 8, 1, 3, 2, 5]
final: [9, 7, 8, 1, 3, 2, 5] válida=True


Ordem crescente (7)
inicial: [1, 2, 3, 4, 5, 6, 7]
sift-down i=2: trocas=[(2, 6)] -> [1, 2, 7, 4, 5, 6, 3]
sift-down i=1: trocas=[(1, 4)] -> [1, 5, 7, 4, 2, 6, 3]
sift-down i=0: trocas=[(0, 2), (2, 5)] -> [7, 5, 6, 4, 2, 1, 3]
final: [7, 5, 6, 4, 2, 1, 3] válida=True


Ordem decrescente (7)
inicial: [7, 6, 5, 4, 3, 2, 1]
sift-down i=2: trocas=[] -> [7, 6, 5, 4, 3, 2, 1]
sift-down i=1: trocas=[] -> [7, 6, 5, 4, 3, 2, 1]
sift-down i=0: trocas=[] -> [7, 6, 5, 4, 3, 2, 1]
final: [7, 6, 5, 4, 3, 2, 1] válida=True


Array arbitrário (10)
inicial: [4, 10, 3, 5, 1, 8, 9, 2, 7, 6]
sift-down i=4: trocas=[(4, 9)] -> [4, 10, 3, 5, 6, 8, 9, 2, 7, 1]
sift-down i=3: trocas=[(3, 8)] -> [4, 10, 3, 7, 6, 8, 9, 2, 5, 1]
sift-down i=2: trocas=[(2, 6)] -> [4, 10, 9, 7, 6, 8, 3, 2, 5, 1]
sift-down i=1: trocas=[] -> [4, 10, 9, 7, 6, 8, 3, 2, 5, 1]
sift-down i=0: trocas=[(0, 1), (1, 3), (3, 8)] -> [10, 7, 9, 5, 6, 8, 3, 2, 4, 1]
final: [10, 7, 9, 5, 6, 8, 3, 2, 4, 1] válida=True


Ordem crescente (15)
inicial: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
sift-down i=6: trocas=[(6, 14)] -> [1, 2, 3, 4, 5, 6, 15, 8, 9, 10, 11, 12, 13, 14, 7]
sift-down i=5: trocas=[(5, 12)] -> [1, 2, 3, 4, 5, 13, 15, 8, 9, 10, 11, 12, 6, 14, 7]
sift-down i=4: trocas=[(4, 10)] -> [1, 2, 3, 4, 11, 13, 15, 8, 9, 10, 5, 12, 6, 14, 7]
sift-down i=3: trocas=[(3, 8)] -> [1, 2, 3, 9, 11, 13, 15, 8, 4, 10, 5, 12, 6, 14, 7]
sift-down i=2: trocas=[(2, 6), (6, 13)] -> [1, 2, 15, 9, 11, 13, 14, 8, 4, 10, 5, 12, 6, 3, 7]
sift-down i=1: trocas=[(1, 4), (4, 9)] -> [1, 11, 15, 9, 10, 13, 14, 8, 4, 2, 5, 12, 6, 3, 7]
sift-down i=0: trocas=[(0, 2), (2, 6), (6, 14)] -> [15, 11, 14, 9, 10, 13, 7, 8, 4, 2, 5, 12, 6, 3, 1]
final: [15, 11, 14, 9, 10, 13, 7, 8, 4, 2, 5, 12, 6, 3, 1] válida=True
```

```
build_heap por padrão de entrada (média de 5 execuções; todas as heaps válidas)
| padrão        |     n |   passos |   trocas |   passos/n |
|---------------|-------|----------|----------|------------|
| sorted        |    10 |    22.00 |     8.00 |       2.20 |
| sorted        |   100 |   286.00 |    96.00 |       2.86 |
| sorted        |  1000 |  2974.00 |   992.00 |       2.97 |
| sorted        | 10000 | 29974.00 |  9992.00 |       3.00 |
| reverse       |    10 |     9.00 |     0.00 |       0.90 |
| reverse       |   100 |    99.00 |     0.00 |       0.99 |
| reverse       |  1000 |   999.00 |     0.00 |       1.00 |
| reverse       | 10000 |  9999.00 |     0.00 |       1.00 |
| random        |    10 |    17.80 |     4.80 |       1.78 |
| random        |   100 |   247.20 |    69.20 |       2.47 |
| random        |  1000 |  2601.00 |   730.40 |       2.60 |
| random        | 10000 | 26198.00 |  7420.80 |       2.62 |
| nearly_sorted |    10 |    22.00 |     8.00 |       2.20 |
| nearly_sorted |   100 |   281.20 |    93.00 |       2.81 |
| nearly_sorted |  1000 |  2941.40 |   973.40 |       2.94 |
| nearly_sorted | 10000 | 29554.80 |  9752.60 |       2.96 |
```

Exercício 09 – build_heap × inserção incremental

ex09/__main__.py

```python
# Inserção incremental: O(n log n)
# Cada insert() faz sift-up, que no pior caso percorre log n níveis.
# Para n elementos: n × O(log n) = O(n log n).
# Os primeiros elementos inseridos ocupam posições próximas à raiz e
# podem ter que subir muitos níveis conforme a heap cresce.
# build_heap: O(n)
# Aplica sift-down nos nós internos de baixo para cima.
# A maioria dos nós fica nos níveis inferiores e faz poucas trocas.
# A soma das alturas de todos os nós converge para O(n).
# Métrica: passos = comparações entre elementos + trocas (contador da
BinaryHeap).

def compare(pattern: str, n: int) -> tuple:
    array = generate_vector(n, pattern)

    heap_insert = BinaryHeap()
    for v in array:
        heap_insert.insert(v)

    heap_build, _ = BinaryHeap.build_heap(array, snapshots=False)

    assert sorted(heap_insert.data) == sorted(heap_build.data)
```

```
    return (heap_insert.steps, heap_build.steps,
            heap_insert.swaps, heap_build.swaps,
            heap_insert.steps / (n * math.log2(n)),
            heap_build.steps / n)

rows, headers = benchmark(
    axes={"padrão": ["sorted", "reverse", "random", "nearly_sorted"],
          "n": [10, 100, 1000, 10000]},
    measure=compare,
    metrics=["insert (passos)", "build (passos)",
             "insert (trocas)", "build (trocas)",
             "insert/(n·log2n)", "build/n"],
    samples=5,
)
print_table("Inserção incremental vs build_heap (média de 5 execuções)",
            rows, headers, floatfmt=".2f")

# Discussão dos resultados:
# - sorted (pior caso da inserção): cada novo elemento sobe até a raiz.
#   insert/(n·log2n) cresce de 1.14 para 1.71 e tende a uma constante (~2,
#   uma comparação + uma troca por nível), confirmando O(n log n). Com
#   n=10000, a inserção faz ~7.6x mais passos que o build_heap.
# - build/n fica sempre entre 0.9 e 3.0, para qualquer padrão e tamanho:
#   o custo cresce linearmente com n, confirmando O(n).
# - reverse: o array já é uma heap de máximo, então nenhuma troca acontece
#   e as duas abordagens custam n-1 comparações (O(n), melhor caso).
# - random: a inserção fica bem abaixo do pior caso (insert/(n·log2n)
#   diminui com n), pois um elemento aleatório sobe em média poucos níveis.
#   Mesmo assim o build_heap faz menos passos em todos os tamanhos.
# Conclusão: build_heap nunca é pior que a inserção incremental e, no pior
# caso, a diferença é assintótica: O(n) contra O(n log n).
```

Execução (`ex09/execucao.txt`):

```
$ python -m ex09
Inserção incremental vs build_heap (média de 5 execuções)
| padrão       |     n |   insert (passos) |   build (passos) |   insert (trocas) |   build (trocas) |   insert/(n·log2n) |   build/n |
|--------------|-------|-------------------|------------------|-------------------|------------------|--------------------|-----------|
| sorted       |    10 |             38.00 |            22.00 |             19.00 |             8.00 |               1.14 |      2.20 |
| sorted       |   100 |            960.00 |           286.00 |            480.00 |            96.00 |               1.44 |      2.86 |
| sorted       |  1000 |          15974.00 |          2974.00 |           7987.00 |           992.00 |               1.60 |      2.97 |
| sorted       | 10000 |         227262.00 |         29974.00 |         113631.00 |          9992.00 |               1.71 |      3.00 |
| reverse      |    10 |              9.00 |             9.00 |              0.00 |             0.00 |               0.27 |      0.90 |
| reverse      |   100 |             99.00 |            99.00 |              0.00 |             0.00 |               0.15 |      0.99 |
| reverse      |  1000 |            999.00 |           999.00 |              0.00 |             0.00 |               0.10 |      1.00 |
| reverse      | 10000 |           9999.00 |          9999.00 |              0.00 |             0.00 |               0.08 |      1.00 |
| random       |    10 |             20.20 |            17.80 |              6.20 |             4.80 |               0.61 |      1.78 |
| random       |   100 |            311.40 |           247.20 |            108.00 |            69.20 |               0.47 |      2.47 |
```

```
| random         |  1000 |          3477.60 |          2601.00 |          1243.40 |           730.40 |             0.35 |      2.60 |
| random         | 10000 |         35347.00 |         26198.00 |         12678.60 |          7420.80 |             0.27 |      2.62 |
| nearly_sorted |    10 |            34.20 |            22.00 |            15.60 |             8.00 |             1.03 |      2.20 |
| nearly_sorted |   100 |           773.80 |           281.20 |           349.20 |            93.00 |             1.16 |      2.81 |
| nearly_sorted |  1000 |          9552.60 |          2941.40 |          4285.40 |           973.40 |             0.96 |      2.94 |
| nearly_sorted | 10000 |         98759.40 |         29554.80 |         44389.20 |          9752.60 |             0.74 |      2.96 |
```

Exercício 10 – Heap sort parcial (k maiores)

ex10/__main__.py

```python
# Algoritmo: build_heap O(n) + k extrações O(k log n) = O(n + k log n).
# Vantagem sobre ordenação completa O(n log n): quando k < n, evita ordenar
# elementos que nunca serão usados. Só os k maiores saem da heap; os n - k
# restantes continuam nela sem nenhuma ordenação entre si.
def k_largest(array: list, k: int) -> tuple:
    # Devolve (k maiores em ordem decrescente, o que restou na heap).
    if not 0 <= k <= len(array):
        raise ValueError(f"k deve estar entre 0 e {len(array)}, recebido
{k}")
    heap, _ = BinaryHeap.build_heap(array, snapshots=False)
    result = []
    for _ in range(k):
        maximum, _ = heap.extract_max()
        result.append(maximum)
    return result, heap.data

def demo(array: list, k: int) -> None:
    print(f"array (n={len(array)}): {array}")
    try:
        result, remaining = k_largest(array, k)
        print(f"k={k} maiores: {result}")
        print(f"restante (sem ordem): {remaining}\n")
    except ValueError as error:
        print(f"k={k} recusado: {error}\n")

small_array = [5, 3, 8, 1, 7, 2, 9, 4, 6]
large_array = list(range(1, 21))

# k < n: seleção parcial, o array não é ordenado por completo
demo(small_array, 1)
demo(small_array, 3)
demo(small_array, 5)
demo(large_array, 5)
demo(large_array, 10)
```

```
# Caso-limite k = n: todos os elementos são extraídos, o que equivale a um
# heap sort completo, O(n log n). É o maior k válido, e não o uso pretendido
# da seleção parcial.
demo(small_array, len(small_array))

# k inválido: recusado com ValueError
demo(small_array, len(small_array) + 1)
demo(small_array, -1)
```

Execução (`ex10/execucao.txt`):

```
$ python -m ex10
array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=1 maiores: [9]
restante (sem ordem): [8, 7, 5, 6, 3, 2, 1, 4]

array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=3 maiores: [9, 8, 7]
restante (sem ordem): [6, 4, 5, 1, 3, 2]

array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=5 maiores: [9, 8, 7, 6, 5]
restante (sem ordem): [4, 3, 2, 1]

array (n=20): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
k=5 maiores: [20, 19, 18, 17, 16]
restante (sem ordem): [15, 11, 14, 9, 10, 13, 7, 8, 4, 1, 2, 12, 6, 3, 5]

array (n=20): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
k=10 maiores: [20, 19, 18, 17, 16, 15, 14, 13, 12, 11]
restante (sem ordem): [10, 9, 7, 8, 5, 6, 3, 2, 4, 1]

array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=9 maiores: [9, 8, 7, 6, 5, 4, 3, 2, 1]
restante (sem ordem): []

array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=10 recusado: k deve estar entre 0 e 9, recebido 10

array (n=9): [5, 3, 8, 1, 7, 2, 9, 4, 6]
k=-1 recusado: k deve estar entre 0 e 9, recebido -1
```

Exercício 11 – Análise de complexidade empírica

ex11/__main__.py

```python
# Métrica: passos = comparações entre elementos + trocas.
# Tempo: time.perf_counter() em volta apenas da operação medida (a montagem
# da heap de entrada fica fora da medição).
# Pior caso: array em ordem crescente.
# insert n vezes: cada elemento sobe até a raiz -> O(n log n)
# build_heap: sift-down de baixo para cima -> O(n)
# extract_max n vezes: cada extração faz sift-down completo -> O(n log n)
# contains: varredura linear, elemento ausente -> O(n)
# delete: busca linear até o último índice -> O(n)

def build_by_insertion(array: list) -> BinaryHeap:
    heap = BinaryHeap()
    for v in array:
        heap.insert(v)
    heap.reset_counters()
    return heap

def op_insert(array):
    heap = BinaryHeap()
    start = time.perf_counter()
    for v in array:
        heap.insert(v)
    return heap.steps, time.perf_counter() - start

def op_build(array):
    start = time.perf_counter()
    heap, _ = BinaryHeap.build_heap(array, snapshots=False)
    return heap.steps, time.perf_counter() - start

def op_extract_max(array):
    heap, _ = BinaryHeap.build_heap(array, snapshots=False)
    heap.reset_counters()
    start = time.perf_counter()
    for _ in range(len(array)):
        heap.extract_max()
    return heap.steps, time.perf_counter() - start

def op_contains(array):
```

```
    heap = build_by_insertion(array)
    start = time.perf_counter()
    heap.contains(-1)  # pior caso: elemento ausente, percorre tudo
    return heap.steps, time.perf_counter() - start

def op_delete(array):
    heap = build_by_insertion(array)
    target = heap.data[-1]  # pior caso: valor no último índice do array
interno
    start = time.perf_counter()
    heap.delete(target)
    return heap.steps, time.perf_counter() - start

OPERATIONS = {
    "insert": (op_insert, "n·log2n", lambda n: n * math.log2(n)),
    "build_heap": (op_build, "n", lambda n: n),
    "extract_max": (op_extract_max, "n·log2n", lambda n: n * math.log2(n)),
    "contains": (op_contains, "n", lambda n: n),
    "delete": (op_delete, "n", lambda n: n),
}

def measure(operation: str, n: int) -> tuple:
    function, _, reference = OPERATIONS[operation]
    steps, seconds = function(generate_vector(n, "sorted"))
    ref = reference(n)
    return steps, ref, steps / ref, seconds * 1000

rows, headers = benchmark(
    axes={"operação": list(OPERATIONS), "n": [100, 500, 1000, 5000, 10000]},
    measure=measure,
    metrics=["passos", "referência", "passos/ref", "tempo (ms)"],
    samples=5,
)
for row in rows:
    row.insert(2, OPERATIONS[row[0]][1])  # coluna com a fórmula de
referência
headers.insert(2, "teoria")
print_table("Complexidade empírica (entrada crescente, média de 5
execuções)",
            rows, headers, floatfmt=".3f")
```

```
# Comparação com a análise teórica:
# - Se a operação tem a complexidade prevista, passos/ref se estabiliza em
#   uma constante à medida que n cresce.
# - insert: passos/ref sobe de 1.44 para 1.71 e desacelera. A constante
tende
#   a 2 porque cada nível percorrido custa uma comparação + uma troca. O(n
log n).
# - build_heap: passos/ref converge para 3.0 (n=100 -> 2.86, n=10000 ->
3.00).
#   Custo linear, O(n), mesmo no pior caso.
# - extract_max (n extrações): passos/ref sobe de 2.01 para 2.53 e
desacelera,
#   pois cada nível do sift-down custa até duas comparações + uma troca. O(n
log n).
# - contains e delete: passos/ref = 1.000 exato, pois o pior caso visita
#   todos os n elementos. O(n).
# - Tempo: acompanha a tendência dos passos, mas varia entre execuções e
máquinas (ruído do sistema).
# Por isso os passos, que são determinísticos, são a métrica principal
```

Execução (`ex11/execucao.txt`):

```
$ python -m ex11
Complexidade empírica (entrada crescente, média de 5 execuções)
| operação    |     n | teoria    |     passos |   referência |   passos/ref |   tempo (ms) |
|-------------|-------|-----------|------------|--------------|--------------|--------------|
| insert      |   100 | n·log2n   |    960.000 |      664.386 |        1.445 |        0.174 |
| insert      |   500 | n·log2n   |   6996.000 |     4482.892 |        1.561 |        1.277 |
| insert      |  1000 | n·log2n   |  15974.000 |     9965.784 |        1.603 |        3.384 |
| insert      |  5000 | n·log2n   | 103644.000 |    61438.562 |        1.687 |       20.208 |
| insert      | 10000 | n·log2n   | 227262.000 |   132877.124 |        1.710 |       56.270 |
| build_heap  |   100 | n         |    286.000 |      100.000 |        2.860 |        0.070 |
| build_heap  |   500 | n         |   1476.000 |      500.000 |        2.952 |        0.443 |
| build_heap  |  1000 | n         |   2974.000 |     1000.000 |        2.974 |        0.853 |
| build_heap  |  5000 | n         |  14977.000 |     5000.000 |        2.995 |        5.387 |
| build_heap  | 10000 | n         |  29974.000 |    10000.000 |        2.997 |       13.862 |
| extract_max |   100 | n·log2n   |   1336.000 |      664.386 |        2.011 |        0.313 |
| extract_max |   500 | n·log2n   |  10135.000 |     4482.892 |        2.261 |        2.317 |
| extract_max |  1000 | n·log2n   |  23318.000 |     9965.784 |        2.340 |        5.254 |
| extract_max |  5000 | n·log2n   | 153082.000 |    61438.562 |        2.492 |       53.343 |
| extract_max | 10000 | n·log2n   | 336443.000 |   132877.124 |        2.532 |      104.408 |
| contains    |   100 | n         |    100.000 |      100.000 |        1.000 |        0.011 |
| contains    |   500 | n         |    500.000 |      500.000 |        1.000 |        0.053 |
| contains    |  1000 | n         |   1000.000 |     1000.000 |        1.000 |        0.107 |
| contains    |  5000 | n         |   5000.000 |     5000.000 |        1.000 |        0.587 |
| contains    | 10000 | n         |  10000.000 |    10000.000 |        1.000 |        1.246 |
| delete      |   100 | n         |    100.000 |      100.000 |        1.000 |        0.008 |
| delete      |   500 | n         |    500.000 |      500.000 |        1.000 |        0.039 |
| delete      |  1000 | n         |   1000.000 |     1000.000 |        1.000 |        0.087 |
```

```
| delete      |  5000 | n         |   5000.000 |     5000.000 |        1.000 |        0.450 |
| delete      | 10000 | n         |  10000.000 |    10000.000 |        1.000 |        0.893 |
```

Exercício 12 – Documentação técnica

Visão Geral da Arquitetura

O projeto implementa uma **heap binária de máximo** encapsulada na classe `BinaryHeap` (arquivo `utils.py`). A validação estrutural fica no método de classe `BinaryHeap.is_valid_heap`, e cada exercício (`ex03` a `ex11`) isola e demonstra empiricamente uma operação ou propriedade específica.

Representação Interna: Array de Nível

Decisão

A heap é armazenada em uma lista Python (`__data`) usando mapeamento de índices para simular a árvore binária quase completa:

- Pai de `i` (`__parent`): `(i - 1) // 2`
- Filho esquerdo de `i` (`__left`): `2 * i + 1`
- Filho direito de `i` (`__right`): `2 * i + 2`

Justificativa

Uma árvore binária quase completa tem a propriedade de que, quando seus nós são numerados em ordem de nível, os índices dos filhos de qualquer nó `i` são exatamente `2i+1` e `2i+2`. Isso elimina completamente a necessidade de ponteiros explícitos.

Consequências:

- A completude da árvore é mantida automaticamente: `append` ao final do array equivale a inserir a próxima folha da esquerda no nível mais baixo incompleto.
- Acesso a pai e filhos é O(1).
- Localidade de cache: elementos adjacentes na árvore ficam próximos na memória.

Propriedade formal preservada

Heap de máximo: para todo nó interno `i`, `data[i] >= data[2i+1]` e `data[i] >= data[2i+2]` (quando os filhos existem).

Essa invariante é a única restrição estrutural. Não há relação de ordem entre elementos de subárvores diferentes, o que distingue a heap de uma BST.

Instrumentação de Passos

Decisão

Dois atributos privados acumulam o custo das operações: `__comparisons` (comparações entre elementos) e `__swaps` (trocas realizadas). Eles são expostos pelas propriedades `comparisons` e `swaps`, e a propriedade `steps` devolve a soma dos dois (os passos). `reset_counters()` zera ambos.

Justificativa

Toda a análise de complexidade do projeto é empírica, não apenas assintótica. O contador permite verificar numericamente que as operações se comportam como O(log n), O(n) ou O(n log n) ao comparar `steps` com as referências `n` e `n * log2(n)` (`ex11`).

Operações e Decisões Algorítmicas

As duas rotinas de reorganização ficam em métodos privados, reutilizados por todas as operações:

- `__sift_up(i)`: sobe o elemento da posição `i` enquanto for maior que o pai. Usado por `insert` e `delete`.
- `__sift_down(i)`: desce o elemento da posição `i`, trocando com o maior filho, até nenhum filho ser maior. Usado por `extract_max`, `delete` e `build_heap`.

Ambos devolvem a lista de trocas realizadas e atualizam os contadores. Centralizar essa lógica evita duplicação e garante que toda operação restaure a invariante da mesma forma.

insert(value) – O(log n)

Algoritmo:

1. Acrescentar valor ao final do array (nova folha).
2. Comparar com o pai, se maior, trocar e subir.
3. Repetir até chegar à raiz ou o pai ser maior.

extract_max() – O(log n)

Algoritmo:

1. Salvar raiz.
2. Mover última folha para a raiz.
3. Comparar com o maior filho, se menor, trocar e descer.
4. Repetir até nenhum filho ser maior.

contains(value) – O(n)

Algoritmo: percorrer todos os elementos do array, registrando índices visitados.

delete(value) – O(n)

Algoritmo:

1. Busca linear pelo valor (O(n)).
2. Substituir o elemento encontrado pela última folha.
3. Comparar o valor substituto com o valor removido:
   - Se substituto maior que removido: sift-up (o novo valor viola a invariante com o pai).
   - Se substituto menor que removido: sift-down (o novo valor viola a invariante com os filhos).
   - Se igual: nenhuma reorganização necessária.

build_heap(array) – O(n)

Algoritmo:

1. Copiar o array bruto para o heap.
2. Para cada nó interno, de baixo para cima (índice `(n-2)//2` até 0), aplicar sift-down nesse nó.
3. Folhas (índices maiores que `(n-2)//2`) são ignoradas: já satisfazem a invariante.

É um `@classmethod` porque cria e devolve uma nova instância a partir de um array. O parâmetro `snapshots` (padrão `True`) guarda uma cópia do array após cada sift-down, para a demonstração passo a passo do `ex08`. Cada cópia custa O(n), então as medições (`ex09`, `ex11`) usam `snapshots=False` para que o tempo real também seja O(n).

is_valid_heap(array) – O(n)

Algoritmo: para cada nó `i`, verificar se `array[left(i)] <= array[i]` e `array[right(i)] <= array[i]`. Retornar imediatamente ao encontrar a primeira violação.

É um `@classmethod`, no mesmo padrão do `build_heap`: recebe um array qualquer, cria uma instância a partir dele (`cls(array)`) e usa os métodos de índice `__left` e `__right` dessa instância para verificar cada relação pai-filho. A chamada é `BinaryHeap.is_valid_heap(array)`. A cópia feita por `cls(array)` é O(n), o mesmo custo da própria verificação.

Relação das Decisões com as Propriedades Formais da Heap

- **Completude da árvore**: `append` ao final garante a próxima folha na ordem BFS, `pop()` remove sempre a última folha. Nenhuma operação quebra essa estrutura.

- **Invariante de máximo** (pai >= filhos): toda operação que move elementos termina com sift-up ou sift-down para restaurar a relação pai-filho em toda a cadeia afetada.
- **Raiz e o máximo global**: se cada pai é maior ou igual a seus filhos, o maior elemento não pode estar em nenhum filho, está na raiz.
- **Altura O(log n)**: a completude garante que uma árvore com `n` nós tem altura `floor(log2(n))`, o que limita superiormente o custo de sift-up e sift-down.
- **Ausência de ordem total entre irmãos**: nenhuma operação força relação entre subárvores irmãs. Isso é intencional e é exatamente o que torna `contains` e `delete` O(n).

LINK REPO

https://github.com/matheusmonteiro-al-infnet/edaa_tp1.git